Buổi 3

Sắp xếp bằng phương pháp đổi chỗ

Học phần: Cấu trúc dữ liệu và Giải thuật

Nội dung bài học

1. Khái quát về sắp xếp

1.1 Bài toán sắp xếp và các tiêu chí đánh giá
1.2 Khái niệm nghịch thế và dãy dùng xuyên suốt

2. Sắp xếp đổi chỗ trực tiếp (Interchange Sort)

2.1 Interchange Sort – tư tưởng, thuật toán, cài đặt
2.2 Interchange Sort – minh họa từng lần đi
2.3 Interchange Sort – phân tích

3. Sắp xếp nổi bọt (Bubble Sort)

3.1 Bubble Sort – tư tưởng, thuật toán, cài đặt
3.2 Bubble Sort – minh họa từng lần đi
3.3 Bubble Sort – phân tích và cải tiến

4. Sắp xếp rung (Shaker Sort)

4.1 Shaker Sort – tư tưởng và cài đặt
4.2 Shaker Sort – minh họa và phân tích

5. Sắp xếp nhanh (QuickSort)

5.1 QuickSort – tư tưởng và hàm phân hoạch
5.2 QuickSort – minh họa lần phân hoạch đầu tiên
5.3 QuickSort – cây đệ quy và toàn bộ quá trình
5.4 QuickSort – phân tích độ phức tạp và chọn phần tử chốt

6. Tổng kết và thực hành

6.1 So sánh các thuật toán đổi chỗ và bài thực hành
6.2 Câu hỏi và bài tập

1.1 Bài toán sắp xếp và các tiêu chí đánh giá

Bài toán

Cho dãy M gồm N phần tử. **Sắp xếp** là sắp đặt lại các phần tử theo một thứ tự nhất định của **khóa** (tăng dần hoặc giảm dần). Trong môn học, mặc định sắp **tăng dần**: M[0] ≤ M[1] ≤ … ≤ M[N−1].

- **Vì sao cần sắp xếp?** Dữ liệu có thứ tự giúp **tìm kiếm nhanh** (tìm nhị phân), lập báo cáo, phát hiện trùng lặp, trộn dữ liệu…
- **Phân loại:**
  - **Sắp xếp nội**: dữ liệu nằm trong bộ nhớ trong (buổi 3, 4, 5).
  - **Sắp xếp ngoại**: dữ liệu trên tập tin (buổi 5).
- **Các nhóm phương pháp sắp xếp nội** (theo giáo trình):

Phương pháp	Thuật toán	Buổi
Đổi chỗ	Interchange, Bubble, Shaker, QuickSort	3
Chọn	Selection, HeapSort	4
Chèn	Insertion, chèn nhị phân, ShellSort	4
Trộn	Straight Merge, Natural Merge	5

- **Tiêu chí đánh giá:**
  - Số phép **so sánh** (S).
  - Số phép **hoán vị / gán** (mỗi hoán vị = 3 phép gán).
  - Bộ nhớ phụ.
  - **Tính ổn định (stable)**: hai phần tử có khóa bằng nhau giữ nguyên thứ tự tương đối ban đầu.

Ví dụ 3.1 – Tính ổn định

Sắp danh sách SV theo điểm, các SV đã sắp theo tên trước đó: (An, 8), (Bình, 7), (Chi, 8). Thuật toán **ổn định** cho (Bình, 7), (An, 8), (Chi, 8) – An vẫn trước Chi; thuật toán không ổn định có thể cho (Chi, 8) trước (An, 8).

1.2 Khái niệm nghịch thế và dãy dùng xuyên suốt

- **Nghịch thế**: cặp (i, j) với i < j mà M[i] > M[j]. Dãy đã sắp tăng $\Leftrightarrow$ **không còn nghịch thế** nào. Mọi thuật toán sắp xếp đổi chỗ đều tìm cách **triệt tiêu nghịch thế** bằng hoán vị.

```
void Swap(T &X, T &Y)      // hoán vị: 3 phép gán
{  T Temp = X;
   X = Y;
   Y = Temp;
}
```

Ví dụ 3.2 – Dãy minh họa (N = 8)

Chỉ số	0	1	2	3	4	5	6	7
M	42	23	74	11	65	58	94	36

- Kết quả mong muốn: **11 23 36 42 58 65 74 94**.

Ví dụ 3.3 – Đếm nghịch thế của dãy trên

- 42 đứng trước và lớn hơn: 23, 11, 36 $\Rightarrow$ 3
- 23: 11 $\Rightarrow$ 1; 74: 11, 65, 58, 36 $\Rightarrow$ 4
- 11: 0; 65: 58, 36 $\Rightarrow$ 2; 58: 36 $\Rightarrow$ 1
- 94: 36 $\Rightarrow$ 1; 36: 0
- **Tổng = 12 nghịch thế.** Dãy giảm dần N phần tử có tối đa $N(N-1)/2 = 28$ nghịch thế (trường hợp xấu nhất).

- Mỗi hoán vị **hai phần tử kề nhau** triệt tiêu đúng 1 nghịch thế $\Rightarrow$ Bubble Sort cần đúng **12** hoán vị cho dãy này.

2.1 Interchange Sort – tư tưởng, thuật toán, cài đặt

- **Tư tưởng:** xét phần tử đầu dãy M[i], so sánh với **mọi** phần tử đứng sau M[j] (j > i); nếu M[j] < M[i] (nghịch thế) thì **hoán vị**. Sau lượt i, M[i] là phần tử nhỏ nhất của đoạn M[i..N−1]. Lặp với i = 0..N−2.

```
B1: i = 1                          // lần xử lý đầu tiên
B2: IF (i ≥ N) Thực hiện Bkt
B3: j = i + 1
B4: IF (j > N) Thực hiện B8
B5: IF (M[j] < M[i])
      Swap(M[i], M[j])
B6: j++
B7: Lặp lại B4
B8: i++
B9: Lặp lại B2
Bkt: Kết thúc
```

```
void InterchangeSort(T M[], int N)
{  for (int i = 0; i < N - 1; i++)
      for (int j = i + 1; j < N; j++)
         if (M[j] < M[i])
            Swap(M[i], M[j]);
}
```

- Hai vòng lặp lồng nhau; vòng trong luôn chạy hết dù dãy đã có thứ tự.
- Chi tiết lượt 1 (i = 0) với dãy ví dụ: 42 > 23 ⇒ hoán vị (23, 42, …); 23 > 11 ⇒ hoán vị (11, 42, 74, 23, …); không còn phần tử nào nhỏ hơn 11.

2.2 Interchange Sort – minh họa từng lần đi

Ví dụ 3.4 – M = {42, 23, 74, 11, 65, 58, 94, 36} (phần tử in đậm đã về đúng vị trí sau lượt)

Lượt (i)	M[0]	M[1]	M[2]	M[3]	M[4]	M[5]	M[6]	M[7]	Số hoán vị
Ban đầu	42	23	74	11	65	58	94	36	
1 (i=0)	**11**	42	74	23	65	58	94	36	2
2 (i=1)	11	**23**	74	42	65	58	94	36	1
3 (i=2)	11	23	**36**	74	65	58	94	42	2
4 (i=3)	11	23	36	**42**	74	65	94	58	3
5 (i=4)	11	23	36	42	**58**	74	94	65	2
6 (i=5)	11	23	36	42	58	**65**	94	74	1
7 (i=6)	11	23	36	42	58	65	**74**	94	1

- **Nhận xét:** tổng **12** hoán vị, nhưng các phần tử bị di chuyển **qua lại**: ở lượt 1, 23 được đưa lên đầu dãy rồi lại bị đổi xuống vị trí 3 (vì gặp 11); 42 bị đẩy dần về sau qua nhiều lượt. Interchange hoán vị các phần tử **không kề nhau** nhưng không có chiến lược nên không giảm được số hoán vị.
- Số phép so sánh: 7 + 6 + … + 1 = **28** = N(N−1)/2, **không phụ thuộc** dữ liệu.

2.3 Interchange Sort – phân tích

Trường hợp	Số phép so sánh	Số phép hoán vị	Tình huống
Tốt nhất	$N(N-1)/2$	0	Dãy đã tăng dần
Xấu nhất	$N(N-1)/2$	$N(N-1)/2$	Dãy giảm dần
Trung bình	$N(N-1)/2$	$\approx N(N-1)/4$	Dãy ngẫu nhiên

- **Độ phức tạp:** $O(N^2)$ trong mọi trường hợp.
- **Ổn định? Không**: ví dụ (5a, 5b, 3): lượt 1 hoán vị 5a với 3 ⇒ (3, 5b, 5a) – thứ tự 5a, 5b bị đảo.
- **Bộ nhớ phụ:** $O(1)$ (một biến tạm trong Swap).

Ví dụ 3.5 – Ước lượng với N = 10.000

- Số phép so sánh = $10^4 \cdot (10^4 - 1)/2 \approx \mathbf{5 \cdot 10^7}$.
- Dãy giảm dần: thêm $5 \cdot 10^7$ hoán vị = $1{,}5 \cdot 10^8$ phép gán.
- Máy 10^8 phép/giây ⇒ cỡ **1 – 2 giây**; với $N = 10^6$ ⇒ cỡ **3 – 6 giờ**.

Ưu, nhược điểm

Ưu: dễ hiểu, dễ cài đặt, phù hợp N nhỏ và dùng làm nền để so sánh. **Nhược:** luôn $N(N-1)/2$ phép so sánh kể cả khi dãy đã sắp; phần tử bị di chuyển qua lại nhiều lần.

3.1 Bubble Sort – tư tưởng, thuật toán, cài đặt

- **Tư tưởng:** đi từ **cuối dãy lên đầu**, so sánh **hai phần tử kề nhau**; nếu M[j] < M[j−1] thì hoán vị. Sau mỗi lượt, phần tử nhỏ nhất của đoạn chưa sắp "**nổi**" lên vị trí đầu đoạn – như bọt khí nổi lên mặt nước.

```
B1: First = 1
B2: IF (First ≥ N) Thực hiện Bkt
B3: j = N
B4: IF (j ≤ First) Thực hiện B8
B5: IF (M[j] < M[j-1])
      Swap(M[j], M[j-1])
B6: j--
B7: Lặp lại B4
B8: First++
B9: Lặp lại B2
Bkt: Kết thúc
```

```
void BubbleSort(T M[], int N)
{  for (int I = 0; I < N - 1; I++)
      for (int J = N - 1; J > I; J--)
         if (M[J] < M[J - 1])
            Swap(M[J], M[J - 1]);
}
```

Ví dụ 3.6 – Chi tiết lượt 1 (I = 0)

42 23 74 11 65 58 94 **36** → 36 < 94: hoán vị → 36 < 58: hoán vị → 36 < 65: hoán vị → 36 > 11: giữ → **11** < 74: hoán vị → 11 < 23: hoán vị → 11 < 42: hoán vị ⇒ **11** 42 23 74 36 65 58 94 (6 hoán vị).

3.2 Bubble Sort – minh họa từng lần đi

Ví dụ 3.7 – M = {42, 23, 74, 11, 65, 58, 94, 36}

Lượt (I)	M[0]	M[1]	M[2]	M[3]	M[4]	M[5]	M[6]	M[7]	Số hoán vị
Ban đầu	42	23	74	11	65	58	94	36	
1 (I=0)	**11**	42	23	74	36	65	58	94	6
2 (I=1)	11	**23**	42	36	74	58	65	94	3
3 (I=2)	11	23	**36**	42	58	74	65	94	2
4 (I=3)	11	23	36	**42**	58	65	74	94	1
5 (I=4)	11	23	36	42	**58**	65	74	94	0
6 (I=5)	11	23	36	42	58	**65**	74	94	0
7 (I=6)	11	23	36	42	58	65	**74**	94	0

- **Tổng hoán vị = 6 + 3 + 2 + 1 = 12** = đúng số nghịch thế (ví dụ 3.3) – vì mỗi hoán vị hai phần tử kề nhau triệt tiêu đúng một nghịch thế.
- Ngoài phần tử nhỏ nhất "nổi lên", các phần tử lớn cũng dần "**chìm xuống**" (94 về cuối ngay lượt 1).
- **Nhận xét quan trọng:** sau lượt 4 dãy **đã có thứ tự**, nhưng thuật toán vẫn chạy thêm 3 lượt (lượt 5, 6, 7 không hoán vị nào) ⇒ lãng phí 3 + 2 + 1 = 6 phép so sánh. ⇒ **Cải tiến** ở trang sau.

3.3 Bubble Sort – phân tích và cải tiến

Trường hợp	Số phép so sánh	Số phép hoán vị
Tốt nhất (dãy tăng)	N(N−1)/2	0
Xấu nhất (dãy giảm)	N(N−1)/2	N(N−1)/2
Trung bình	N(N−1)/2	N(N−1)/4

- **Ổn định: Có** – chỉ hoán vị khi M[j] < M[j−1] (nghiêm ngặt), hai phần tử bằng nhau không bao giờ đổi chỗ.
- **Nhược điểm:** không nhận biết dãy đã có thứ tự; phần tử lớn ở đầu dãy di chuyển về cuối **rất chậm** (mỗi lượt chỉ tiến 1 vị trí – "con rùa").

Ví dụ 3.8 – Bubble Sort có cờ đánh dấu

```
void BubbleSort2(T M[], int N)
{  for (int I = 0; I < N - 1; I++)
   {  int CoHoanVi = 0;
      for (int J = N - 1; J > I; J--)
         if (M[J] < M[J - 1])
         {  Swap(M[J], M[J - 1]);
            CoHoanVi = 1;
         }
      if (!CoHoanVi) break;   // lượt không hoán vị ⇒ đã
sắp
   }
}
```

- Dãy ví dụ: dừng sau **lượt 5** (lượt đầu tiên không hoán vị) ⇒ 7+6+5+4+3 = 25 phép so sánh thay vì 28.
- Dãy đã tăng dần: chỉ **N − 1** phép so sánh ⇒ tốt nhất **O(N)**.

4.1 Shaker Sort – tư tưởng và cài đặt

- **Tư tưởng** – cải tiến Bubble Sort theo hai hướng:
  - Mỗi lượt gồm **2 lượt đi**: lượt **xuống** (từ cuối lên đầu) đẩy phần tử nhỏ lên đầu; lượt **lên** (từ đầu xuống cuối) đẩy phần tử lớn về cuối ⇒ khắc phục "con rùa".
  - Ghi nhớ vị trí **hoán vị cuối cùng** k để **thu hẹp** đoạn cần xét [Left, Right] – các phần tử ngoài đoạn đã đúng chỗ.

```
void ShakerSort(T M[], int N)
{  int Left = 0, Right = N - 1, k = N - 1;
   while (Left < Right)
   {  for (int j = Right; j > Left; j--)   // lượt xuống
         if (M[j] < M[j - 1])
         {  Swap(M[j], M[j - 1]);  k = j;  }
      Left = k;
      for (int j = Left; j < Right; j++)    // lượt lên
         if (M[j] > M[j + 1])
         {  Swap(M[j], M[j + 1]);  k = j;  }
      Right = k;
   }
}
```

- **Vì sao Left = k?** Sau lượt xuống, hoán vị cuối cùng xảy ra tại (k−1, k) ⇒ đoạn M[0..k−1] đã có thứ tự và nhỏ hơn phần còn lại.
- **Vì sao Right = k?** Tương tự: hoán vị cuối cùng của lượt lên tại (k, k+1) ⇒ M[k+1..N−1] đã đúng vị trí.
- Nếu một lượt không có hoán vị nào ⇒ Left ≥ Right ⇒ dừng ngay.

Ví dụ 3.9 – M = {42, 23, 74, 11, 65, 58, 94, 36}

Lượt	Hướng	Kết quả sau lượt đi	Left	Right
Ban đầu		42 23 74 11 65 58 94 36	0	7
1	Xuống (7 → 1)	**11** 42 23 74 36 65 58 94	**1**	7
1	Lên (1 → 6)	11 23 42 36 65 58 74 **94**	1	**5**
2	Xuống (5 → 2)	11 23 36 42 58 65 74 94	**3**	5
2	Lên (3 → 4)	11 23 36 42 58 65 74 94 (không hoán vị)	3	**3**
Kết thúc	Left = Right	**11 23 36 42 58 65 74 94**		

- **Phân tích ví dụ:** chỉ **4 lượt đi** (7 + 6 + 4 + 2 = 19 phép so sánh) so với 28 của Bubble Sort; số hoán vị vẫn là **12** (đều là hoán vị kề nhau).
- Lượt lên thứ nhất kết thúc với hoán vị cuối tại (5, 6) ⇒ Right = 5 (74, 94 đã đúng chỗ) – tiết kiệm được 1 phép so sánh ở lượt sau.

Trường hợp	Số phép so sánh	Hoán vị
Tốt nhất (dãy tăng)	N − 1	0
Xấu nhất	N(N−1)/2	N(N−1)/2
Trung bình	O(N²)	N(N−1)/4

- Ổn định: **Có**. Hiệu quả rõ khi dãy **gần có thứ tự**.

5.1 QuickSort – tư tưởng và hàm phân hoạch

- **Tư tưởng (C. A. R. Hoare, 1960) – chia để trị:** chọn một phần tử **X** (chốt, pivot) – thường là phần tử giữa. **Phân hoạch** dãy thành 2 đoạn: đoạn trái gồm các phần tử **≤ X**, đoạn phải gồm các phần tử **≥ X**. Sau đó sắp xếp **đệ quy** từng đoạn.
- **Thuật toán phân hoạch đoạn [Left, Right]:**

```
B1: X = M[(Left + Right)/2]
B2: i = Left;  j = Right
B3: WHILE (M[i] < X)  i++      // tìm phần tử sai chỗ bên
trái
B4: WHILE (M[j] > X)  j--      // tìm phần tử sai chỗ bên
phải
B5: IF (i ≤ j)
      Swap(M[i], M[j]); i++; j--
B6: IF (i ≤ j) Lặp lại B3
B7: // Lúc này: M[Left..j] ≤ X ≤ M[i..Right]
    IF (Left < j)  Phân hoạch đoạn [Left, j]
    IF (i < Right) Phân hoạch đoạn [i, Right]
Bkt: Kết thúc
```

```
void QuickSort(T M[], int Left, int Right)
{  T X = M[(Left + Right)/2];
   int i = Left, j = Right;
   do
   {  while (M[i] < X) i++;
      while (M[j] > X) j--;
      if (i <= j)
      {  Swap(M[i], M[j]);
         i++;  j--;
      }
   } while (i <= j);
   if (Left < j)  QuickSort(M, Left, j);
   if (i < Right) QuickSort(M, i, Right);
}
// Gọi: QuickSort(M, 0, N - 1);
```

- Hai vòng while dừng được vì X có trong đoạn đóng vai trò **cầm canh**.

5.2 QuickSort – minh họa lần phân hoạch đầu tiên

Ví dụ 3.10 – M = {44, 55, 12, 42, 94, 18, 6, 67}, đoạn [0, 7], X = M[3] = 42

Bước	i dừng tại	j dừng tại	Thao tác	Dãy sau thao tác
Ban đầu			i = 0, j = 7	44 55 12 42 94 18 6 67
1	i = 0 (44 ≥ 42)	j = 6 (6 ≤ 42)	Swap(44, 6); i = 1, j = 5	**6** 55 12 42 94 18 **44** 67
2	i = 1 (55)	j = 5 (18)	Swap(55, 18); i = 2, j = 4	6 **18** 12 42 94 **55** 44 67
3	i = 3 (42)	j = 3 (42)	Swap(42, 42); i = 4, j = 2	6 18 12 42 94 55 44 67
Kết thúc	i = 4 > j = 2			Đoạn trái [0, 2]: 6 18 12 – đoạn phải [4, 7]: 94 55 44 67

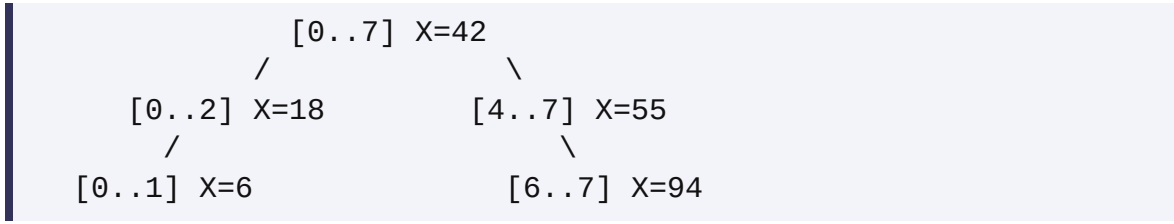
- **Kiểm tra tính chất phân hoạch:** mọi phần tử của đoạn trái {6, 18, 12} ≤ 42; mọi phần tử đoạn phải {94, 55, 44, 67} ≥ 42; 42 nằm **đúng vị trí cuối cùng** (chỉ số 3).
- Chỉ **2 hoán vị thực sự** đã đưa 4 phần tử về đúng phía – QuickSort hoán vị các phần tử ở **xa nhau**, triệt tiêu nhiều nghịch thế một lúc.

Câu hỏi thường gặp

- **Vì sao dùng < X và > X (không phải ≤, ≥)?** Để i, j dừng tại phần tử bằng X ⇒ vòng lặp không chạy ra ngoài mảng, và phân hoạch cân bằng hơn khi có nhiều phần tử trùng.
- **Swap(42, 42)** ở bước 3 là vô hại, giúp i và j vượt qua nhau.

Ví dụ 3.11 – Tiếp tục sắp M = {44, 55, 12, 42, 94, 18, 6, 67}

Lời gọi	Đoạn	X	Kết quả phân hoạch (toàn dãy)	Gọi tiếp
1	[0, 7]	42	6 18 12 **42** 94 55 44 67	[0, 2] và [4, 7]
2	[0, 2]	18	6 12 **18** 42 94 55 44 67	[0, 1] (i = 2 = Right ⇒ không gọi phải)
3	[0, 1]	6	**6** 12 18 42 94 55 44 67	i = 1 ⇒ không gọi tiếp
4	[4, 7]	55	6 12 18 42 44 **55** 94 67	[4, 4]? Left = j = 4 ⇒ không; [6, 7]
5	[6, 7]	94	6 12 18 42 44 55 **67 94**	dừng



- Độ sâu cây đệ quy = 3 ≈ $\log_2 8$ ⇒ trường hợp **gần tốt nhất**.

- **Tổng kết ví dụ:** 5 lời gọi phân hoạch, mỗi "tầng" của cây xử lý tổng cộng ≈ N phần tử ⇒ chi phí ≈ N × (số tầng) = $N \cdot \log_2 N$.
- Mỗi lời gọi tạo một **khung trong ngăn xếp** (stack) ⇒ bộ nhớ phụ O(log N) khi cân bằng.

5.4 QuickSort – phân tích độ phức tạp và chọn phần tử chốt

Trường hợp	Khi nào	Số phép so sánh	Độ phức tạp
Tốt nhất	Mỗi lần X là **trung vị** ⇒ chia đôi đều	$\approx N \cdot \log_2 N$	O(N log N)
Trung bình	Dữ liệu ngẫu nhiên	$\approx 1{,}39 \cdot N \cdot \log_2 N$	O(N log N)
Xấu nhất	Mỗi lần X là **max/min** của đoạn ⇒ một đoạn rỗng	$\approx N^2/2$	$O(N^2)$

- **Truy hồi:** tốt nhất T(N) = 2T(N/2) + cN ⇒ N log N; xấu nhất T(N) = T(N−1) + cN ⇒ N^2.

Ví dụ 3.12 – Trường hợp xấu nhất

Chọn chốt là **phần tử đầu** M[Left] với dãy **đã tăng dần** 1 2 3 … N: mỗi lần chỉ tách ra 1 phần tử ⇒ N tầng đệ quy, $\approx N^2/2$ so sánh và có thể **tràn ngăn xếp** khi N lớn.

- **Các cách chọn chốt:**
  - Phần tử **giữa** (giáo trình) – tốt với dãy đã/gần sắp.
  - **Ngẫu nhiên**: X = M[Left + rand() % (Right − Left + 1)].
  - **Trung vị của 3**: median(M[Left], M[Mid], M[Right]).
- **Cải tiến thực tế:** đoạn nhỏ (≤ 10 – 20 phần tử) chuyển sang **Insertion Sort**; gọi đệ quy cho đoạn **nhỏ hơn** trước để ngăn xếp chỉ O(log N).
- **Ổn định? Không** (hoán vị xa).

6.1 So sánh các thuật toán đổi chỗ và bài thực hành

Thuật toán	So sánh – tốt nhất	So sánh – xấu nhất	Hoán vị – xấu nhất	Ổn định	Bộ nhớ phụ
Interchange	N(N−1)/2	N(N−1)/2	N(N−1)/2	Không	O(1)
Bubble (có cờ)	N − 1	N(N−1)/2	N(N−1)/2	Có	O(1)
Shaker	N − 1	N(N−1)/2	N(N−1)/2	Có	O(1)
QuickSort	≈ N $\log_2$N	≈ $N^2/2$	≈ $N^2/2$ (hiếm)	Không	O(log N) – O(N)

Thực hành – Cài đặt BubbleSort, QuickSort và in trạng thái mảng sau mỗi lần đi

```
void InMang(T M[], int N, const char *TieuDe)
{  printf("%-12s:", TieuDe);
   for (int k = 0; k < N; k++) printf("%4d", M[k]);
   printf("\n");
}
// Trong BubbleSort: sau vòng for J, gọi InMang(M, N, "Luot I")
// Trong QuickSort: sau vòng do-while, in Left, Right, X và mảng
```

- Yêu cầu: (1) chạy với dãy ví dụ 3.2 và 3.10, đối chiếu kết quả từng lượt với bảng trong slide; (2) thêm biến đếm SoSanh, HoanVi; (3) chạy với dãy **tăng**, **giảm**, **ngẫu nhiên** N = 1.000 và lập bảng so sánh 4 thuật toán.

6.2 Câu hỏi và bài tập

Câu hỏi

- 1) Trình bày tư tưởng của Interchange Sort, Bubble Sort, Shaker Sort, QuickSort. Thuật toán nào ổn định?
- 2) Vì sao số hoán vị của Bubble Sort bằng đúng số nghịch thế của dãy?
- 3) Khi nào QuickSort rơi vào trường hợp xấu nhất? Nêu hai cách khắc phục.

Bài tập (có đáp số để tự kiểm tra)

- 1) Mô phỏng Interchange Sort và Bubble Sort trên dãy **{15, 8, 22, 3, 9}**: in dãy sau mỗi lượt và đếm số hoán vị. → **Đáp số Bubble:** lượt 1: 3 15 8 22 9; lượt 2: 3 8 15 9 22; lượt 3: 3 8 9 15 22 ⇒ tổng **6** hoán vị (= 6 nghịch thế).
- 2) Mô phỏng QuickSort (chốt giữa) trên dãy **{42, 23, 74, 11, 65, 58, 94, 36}**: lần phân hoạch đầu X = 11. → **Gợi ý:** sau phân hoạch 1 dãy thành 11 23 74 42 65 58 94 36, đoạn trái rỗng, đoạn phải [1, 7] – một lần chia "lệch".
- 3) Sửa các thuật toán để sắp xếp **giảm dần**; sắp xếp mảng struct SinhVien theo DTB giảm dần, cùng DTB thì theo tên tăng dần.
- 4) Cài đặt QuickSort **không đệ quy** dùng ngăn xếp tự quản lý (lưu cặp Left, Right) – chuẩn bị cho buổi 9.